TECHNICAL DESIGN · HLD & LLD

Khazana v2
Online & Automated

Architecture, modules, login, data storage, external APIs and costs for adding automatic, multi-device money tracking to the offline-first Khazana core.

Status: Draft 0.3: solo build plan, .NET microservices, Phase 1 built
Date: 18 October 2026 · **Prepared by:** Sandhoshsivan M
Audience: Sandhoshsivan M as solo developer, plus future reviewers and collaborators

Contents

1. Summary 3
2. Goals, non-goals and principles 5
3. What v1 already gives us 6
4. HLD: system architecture 8
5. HLD: frontend and app sections 10
6. Login, identity and encryption keys 12
7. HLD: backend 14
8. LLD: backend 16
9. LLD: automatic capture engine (device) 22
10. Data storage 26
11. Module-by-module implementation 28
12. External APIs and their cost 30
13. Cost: what you will actually spend 32
14. Security architecture and compliance 35
15. Solo build plan: roadmap, learning, folders 39
16. Decisions 43
17. Risks 44
Appendix A. Glossary 45
Appendix B. Sources 46

How to use this document. Sections 1–3 explain what we're building and why. Sections 4–10 are the design: read 4–6 for the big picture, then the part you own. Sections 11–14 cover scope, cost and security. **Section 15 is the step-by-step solo build plan:** dated phases, what to learn when, and the full folder structure. Section 16 lists the open decisions. Every cost is an estimate unless a source is linked; get vendor quotes before committing.

1 Summary

Khazana v2 adds automatic data capture and multi-device sync to the existing offline-first app. The phone stays the source of truth, and the server is a thin, encrypted relay that never stores readable financial data.

30–42
person-months of work (estimate)

~27 months
solo, part-time, AI-assisted (Oct 2026 – Dec 2028)

₹8–28 L
cash spend before the v2 launch (your time not counted)

7
phases, each one shippable

What v2 does

- **Captures transactions automatically** from bank SMS (Android), bank-app notifications (Android, already in v1), card alert emails (Gmail), receipts and screenshots (on-device OCR), and bank data through the RBI Account Aggregator (AA) framework.
- **Posts confident entries straight to the ledger** with a notification and Undo. Doubtful entries go to a **Review inbox** where the user approves, edits the amount (for example ₹1,000 → ₹10,000) or rejects. Pending entries never change a balance.
- **Covers the whole financial picture:** bank accounts, credit and debit cards, FD/RD, mutual funds, ETFs, NSE/BSE stocks, bonds, NPS, EPF/EPS/PPF, insurance, plus credit report and card-offer savings.
- **Syncs every device** (Android, iOS, macOS, web) with end-to-end encryption.
- **Keeps every v1 screen** and the existing theme, with smoother motion, prototyped before implementation.

Key design choices

Area	Recommendation
Clients	Keep **Flutter** as the primary app (Android, iOS, macOS) and the **Next.js web app + Tauri** for web and desktop. No rewrite.
Backend	**Ten .NET 10 microservices** behind a **YARP gateway**, talking through **RabbitMQ** events, run locally with **.NET Aspire** and in AWS Mumbai on **ECS Fargate**. A database per service (decision D1: chosen to learn microservices).
Login	**Passkeys (preferred), Google and Apple sign-in**, with device-bound short-lived tokens and rotating refresh tokens. The existing PIN/biometric vault lock stays. No phone-OTP login.
Privacy	End-to-end encrypted sync. SMS, notifications, Gmail and OCR are processed **on the device**. AA and credit data are processed on the server only in memory, then sealed to the user's key and deleted once delivered.
Security	**Top-tier baseline (§14):** OWASP MASVS L2 + Resilience and ASVS L3, passkeys, device-bound tokens, step-up biometrics, app attestation, HSM-held keys, CERT-In VAPT, ISO 27001 target. Card numbers, CVV, UPI PIN and bank passwords are never stored.
Automation	One capture pipeline (filter → parse → match account → dedupe → score → route) feeding the existing double-entry ledger.

Phase 1 is built. Automatic capture on Android — parser v2, routing with confidence scores, the review inbox with approve, edit, reject and Undo, and bank SMS in a `sideload` build — is in PR #28 with CI green. §3 lists what was built and where it differs from the original plan. What remains for Phase 1 is a test on a real phone, the Cards prototype (Slice 1.7) and the PRD update.

Biggest dependencies. (1) Account Aggregator and credit reports both need a **regulated partner and a registered company**. (2) Reading SMS on Android needs **Google Play's SMS permission approval**. (3) Reading Gmail needs Google's **restricted-scope verification**, including a yearly security assessment. These set the timeline more than the code does.

2 Goals, non-goals and principles

The main purpose is to cut manual data entry to near zero without giving up the privacy that defines Khazana.

Goals

- **G1 Automatic capture.** Most transactions arrive with no typing.
- **G2 One app, every device.** Same data on phone, laptop and web.
- **G3 Complete net worth.** Banks, cards, deposits, market investments, retirement schemes and insurance.
- **G4 Card intelligence.** Credit report, card due dates, and "you can save ₹X with the cards you already have".
- **G5 Smooth UI.** Existing theme with better motion and transitions.

Non-goals

- Lending, selling cards or loans, ads, or selling user data.
- Locking any feature that's free today. How v2 earns money is open decision D13.
- Trading or placing orders. We show holdings, not a broker.
- Tick-by-tick live prices. End-of-day prices are enough.
- A separate online-only app. v2 is the same app with an online layer.

Principles

#	Principle	What it means in practice
P1	Offline-first	The device database is the source of truth. Every screen works offline, and the server holds an encrypted copy. If the server shuts down, the app keeps working.
P2	No readable financial data stored on the server	Sync records are encrypted on the device. AA and credit payloads exist in plaintext only in server memory while they're processed.
P3	Balances come only from posted entries	Pending or flagged captures never change a balance.
P4	Every automation can be undone	Auto-posted entries can be undone, which creates reversing postings rather than deleting anything. User edits keep the original parsed value.
P5	Reuse v1	Reuse the domain logic, the Dart ↔ TypeScript parity tests, the Decimal money type, the encryption and the sync merge. Extend them, don't rewrite them.
P6	Least data	Raw SMS and email text is discarded right after parsing (as v1 already does). Only the extracted fields are kept.

> **Deliberate policy change from v1.** v1 never saves a capture without review (`capture_pipeline.dart` : "Nothing is ever auto-committed"). v2 auto-posts high-confidence captures, always with a notification and Undo. An **"Always review captured transactions"** setting keeps the v1 behaviour for users who want it. The PRD needs updating to match.

3 What v1 already gives us

Much of the hardest groundwork already exists, which reduces the v2 effort. Paths refer to the `feature/safety-net-and-hardening` branch.

Capability	Where it lives today	Use in v2
Flutter app	Riverpod, go_router, Drift over SQLCipher (AES-256), 22 tables, schema v6	REUSE Primary client. Add Drift migration v7.
Web + desktop app	`webapp/` : Next.js 16, React 19, Zustand, Dexie + WebCrypto, framer-motion, Tauri 2	REUSE Web client, which gets sync and read-mostly views.
Notification capture	`android/…/KhazanaNotificationListener.kt` , `CaptureBus.kt` , `capture_channel.dart`	EXTEND Add an SMS receiver on the same bus.
Capture pipeline and review inbox	`capture_pipeline.dart` , `pending_capture` entity and DAO, `capture_inbox_screen.dart` , `review_queue_banner.dart`	EXTEND Add confidence scoring, account matching, edit-and-approve and auto-post.
Message parser	`notification_parser.dart` / `notificationParser.ts` with bank profiles, balance stripping and shared test fixtures	EXTEND Extract account last-4, UTR and available balance. Templates become server-updatable.
Duplicate detection	`bank_fingerprint.dart` (SHA-256 fingerprint), `idempotency.ts`	EXTEND Cross-source matching (UTR, amount and time window).
Double-entry ledger	Postings since schema v3, `ledger_writer.dart` , `account_ledger.dart`	REUSE Auto-posts and reversals go through the same writer.
Sync merge	`sync_merge.dart` / `syncMerge.ts` : last-write-wins on `updatedAt` , 90-day tombstones, symmetric	REUSE Runs on the client over encrypted records relayed by the server.
Encryption	PBKDF2-HMAC-SHA256 600k, SQLCipher, AES-256-GCM backups, biometric gate	REUSE Base of the key hierarchy (§6).
Investments	Lot-level instruments, trades, prices and dividends. `amfi_nav_provider.dart` , XIRR, broker and lot CSV import	EXTEND Holdings from AA, FD/RD, NPS, EPF/EPS/PPF.
Market quotes	`webapp/netlify/functions/quote.ts` (ticker symbols only)	EXTEND Moves into the backend's market-data module.
Insurance, goals, liabilities, budgets, reports, health score, safety net	Their feature folders under `lib/features/`	REUSE Unchanged, now fed by automatic data.

Phase 1 as built (PR #28)

Built on `feat/phase1-auto-capture` ; 55 new Flutter tests, Dart and TypeScript parsers pinned by one shared fixture. Where the build differs from the earlier plan, the build wins and this document now follows it.

Piece	What it does	Code
Parser v2	Reference/UTR, account last-4, available balance, instrument, kind (refund and card bill are not income). Rejects OTPs, promotions, declined and failed transactions. Fingerprint uses reference + direction, or the balance, so two identical purchases stay two.	`lib/domain/services/notification_parser.dart`, `webapp/src/domain/notificationParser.ts`, `test/fixtures/parser_cases.json`
Database v7	Captures keep a status after a decision (v1 deleted them, so a re-delivered alert came back as a new draft); routing fields; `CaptureAccountLinks`; indexed dedupe lookup. Migration guarded per column.	`lib/data/models/tables.dart`, `app_database.dart`
Router	0–100 score plus forced-review rules; auto-post at 80 or more (§9.3).	`lib/domain/services/capture_router.dart`
Pipeline and review	Auto-post, approve, edit-and-approve, reject, undo back to the inbox; learns merchant → category and suffix → account. Messages processed one at a time.	`lib/features/capture/services/capture_pipeline.dart`
Review inbox	"To review" with reasons, editable amount, category and account; "Recently added" with Undo; app-wide "Added · Undo" snackbar; "Always review" switch.	`lib/features/capture/screens/capture_inbox_screen.dart`
Android SMS	`play` and `sideload` flavors. Sideload reads bank SMS (live, catch-up on unlock, 90-day import); personal senders dropped natively. The Play build has no SMS permission and no SMS code.	`android/app/src/sideload/`, `sms_catch_up.dart`

Two findings from building it. (1) **Security:** v1's balance regex backtracked on long runs of spaces in notification text, which any app can post — 147 ms at 1,000 spaces and no length limit. Input is now length-capped and whitespace-collapsed before any pattern runs. (2) **Play policy:** v1's comments said Play forbids reading bank SMS; Play's policy actually lists an "SMS-based money management" exception, granted after review. Until Khazana is approved, SMS lives only in the `sideload` build.

4 HLD: system architecture

Devices do the processing. In the cloud, ten small .NET services behind one gateway handle login, encrypted sync and the connectors that legally must run on a server (Account Aggregator, credit bureau).

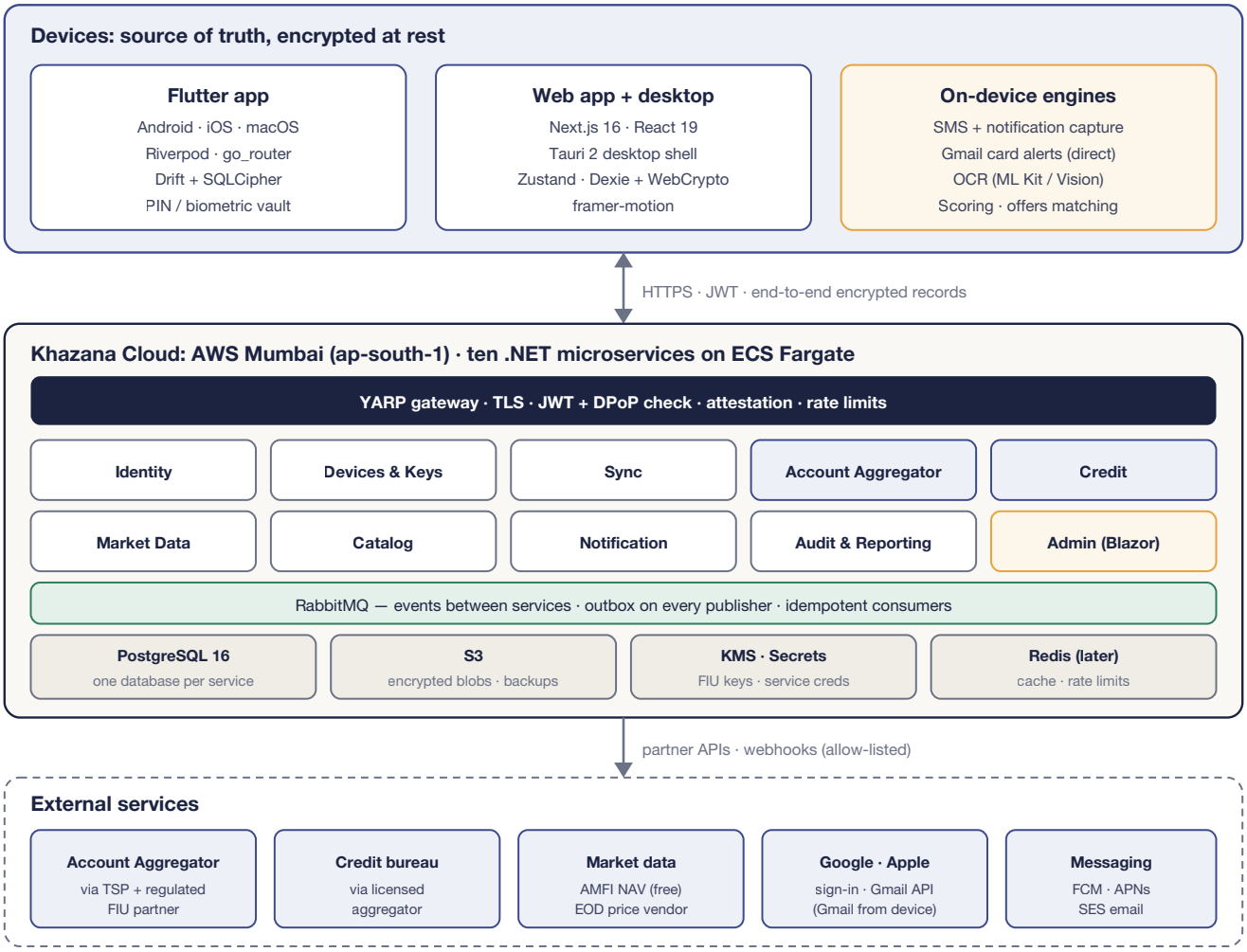


Figure 1. System context. Solid boxes are ours, dashed are third parties. The cloud is ten .NET services behind one gateway; they reach each other only through RabbitMQ events, never each other's databases. Gmail is called directly from the device, so emails never pass through our servers.

Where each kind of data is processed

Data	Processed on	Why
Bank SMS, app notifications	Device	Available only on the phone. Raw text is discarded right after parsing.
Card alert emails (Gmail)	Device	The device calls the Gmail API with the user's own OAuth token, so we never hold mailbox access on a server.
Receipts, screenshots	Device	ML Kit (Android) and Apple Vision (iOS) do OCR offline at no cost.
Account Aggregator data	Server, in memory only	The AA protocol delivers data to the FIU's server key. We decrypt, normalise, seal to the user's device key, deliver, then delete.

Data	Processed on	Why
Credit report	Server, in memory only	Bureau APIs need server credentials. Same seal-and-deliver pattern.
Card offers, prices, parser rules	Server (public data)	Contain no user data. Matching against the user's cards and spending happens on the device.
Everything synced between devices	Device (encrypt) → server (store) → device (decrypt)	End-to-end encrypted. The server stores ciphertext and version numbers only.

Deployment

- **Region:** AWS ap-south-1 (Mumbai), keeping data in India. **One ECS Fargate service per microservice** (two small tasks each in production), plus the gateway.
- **Managed data services:** RDS PostgreSQL (Multi-AZ in production) holding **one database and one login per service** — separate instances only when load demands it; Amazon MQ for RabbitMQ (or self-hosted on Fargate to save cost early); S3 with versioning; KMS and Secrets Manager. Redis only when caching needs it.
- **While building:** nothing in the cloud. `.NET Aspire` runs every service, Postgres and RabbitMQ on the laptop with one command, at ₹0.
- **Edge:** CloudFront plus WAF in front of the API, with rate limits per IP and per user.
- **Environments:** `dev`, `staging` (wired to the AA and bureau sandboxes) and `prod`, all defined in infrastructure-as-code (Terraform or CDK).
- **CI/CD:** extend the existing GitHub Actions (`.github/workflows/ci.yml`) with **one backend job per service**, triggered only by changes under its folder (build, tests, contract tests, container image, deploy to staging on merge, manual promotion to prod).
- **Web hosting:** the web app keeps its static Netlify deploy. The Netlify quote function moves into the backend.

5 HLD: frontend and app sections

Flutter remains the main app, and the web app follows. Both keep the existing layered design and theme. v2 adds three new areas (Accounts, Cards, Connections) and upgrades the Review inbox.

Frontend stack

Concern	Flutter app (primary)	Web app (Next.js + Tauri)
State	Riverpod 2 (existing)	Zustand (existing)
Navigation	go_router (existing)	Next.js App Router (existing)
Local data	Drift + SQLCipher (existing), migration v7	Dexie encrypted blobs (existing)
API client	NEW `data/remote/` : typed client generated from OpenAPI, retry with backoff, idempotency keys	NEW Same OpenAPI-generated TypeScript client
Sync engine	NEW `core/sync/` : push/pull loop over existing `sync_merge.dart`	NEW Same, over `syncMerge.ts`
Motion	Flutter's built-in animation APIs (implicit animations, `AnimationController` , Hero, page transitions). Add Rive only if design delivers Rive files.	framer-motion (existing)
Charts	Custom painters in `presentation/charts/` (existing, no chart package)	SVG components with pixel parity (existing)
Native Android	BUILT Kotlin `SmsReceiver` and inbox catch-up in the `sideload` flavor, on the existing `CaptureBus`	n/a
OCR	NEW Google ML Kit text recognition (Android), Apple Vision (iOS), on-device	Not in scope for web
Gmail	NEW Google Sign-In with the `gmail.readonly` scope, Gmail REST queries filtered to bank senders	Not in scope for web
Push	NEW FCM (Android), APNs (iOS). Local notifications stay as they are.	Web Push (optional)

Theme and motion

We keep the existing design tokens (`app_tokens.dart` , `khazana_colors.dart` , `globals.css`) and the Zentra kit. Reference apps (such as Fold's card stack and tab transitions) inform **motion only**, not colours or components. Before any screen changes, design produces a **clickable prototype** for review. Motion budget: 150–300 ms transitions, spring curves for cards, and no animation that delays data.

App sections

Section	Screens	v2 changes	Data source
Home	Dashboard, balance hero, wallet stack, insights, latest transactions	Linked-account strip, review badge, sync status, card due-date cards	Ledger (all sources)

Section	Screens	v2 changes	Data source
Money	Transactions, add, search, budget, calendar, recurring	Source chip on each row (SMS, email, AA, scan, manual), edit history	Ledger
Accounts NEW	Bank accounts list, account detail, link-account flow	AA linking, last sync time, consent status, balance-drift warning	AA plus SMS plus manual
Cards NEW	My cards (credit and debit), card detail, Credit report, Offers and savings	Limit, utilisation, statement and due dates, savings insights	Gmail, SMS, bureau, offers catalog
Invest	Holdings, analytics, add lot, import	MF, ETF, stocks and bonds from AA. FD/RD, NPS. EPF, EPS and PPF (manual or passbook PDF)	AA plus prices plus manual
Protect	Insurance, Safety net	Policies from AA where available, policy PDF upload	AA plus manual
Plan	Goals, liabilities, reports, tax, forecast	Fed by automatic data, no structural change	Ledger
Health	Score	Unchanged	Ledger
Review inbox UPGRADE	Pending list, item detail, bulk actions	Approve, edit-and-approve, reject, confidence reasons, Undo history	Capture engine
Connections NEW	SMS access, notification access, Gmail link, AA consents, devices	Grant and revoke each source, see last activity	Device plus server
Account and security NEW	Sign-in, devices, recovery key, export, delete account	Device pairing by QR, remote logout	Server plus device

Navigation (mobile)

Bottom bar: **Home · Money · Cards · Invest · More**. "More" holds Accounts, Protect, Plan, Health, Review inbox, Connections and Settings. Each section uses top tabs for its sub-screens, the pattern in the reference app (for example Cards → My cards · Credit report · Offers). Desktop and web keep the sidebar.

Offline behaviour

- Reading, adding, editing, reviewing and capturing all work offline.
- Linking an account, "fetch now", credit report and offers refresh need the network. They queue and show a clear "waiting for connection" state.
- Users who never sign in get v1 behaviour: local only, no account.

6 Login, identity and encryption keys

There are two separate locks. The online account proves who you are to the server, and the existing vault PIN unlocks your data on the device. The server can never read data even with a valid login.

Sign-in methods

Method	When	Notes
Sign in with Google	Launch	Most Android users have it. The app verifies Google's ID token, and the backend issues our own tokens.
Sign in with Apple	Launch (iOS)	Apple requires it (or an equivalent) when an iOS app offers third-party sign-in.
Email magic link	Launch	Fallback for users without Google or Apple. Sent via SES.
Passkeys	Launch (preferred)	Phishing-resistant, no passwords to leak. Offered first wherever the device supports it.
Phone OTP	**Not used**	Every SMS costs money, and SIM-swap fraud is a known attack on Indian finance apps.

Recommendation: issue tokens ourselves (verifying Google and Apple ID tokens is a small amount of code with no per-user fees) rather than using a paid identity service. Decision D3.

Tokens and sessions

- **Access token:** JWT signed with ES256, valid 15 minutes, carrying `sub` (user), `did` (device) and `scope`.
- **Refresh token:** random 256-bit, valid 30 days, **rotated on every use** with reuse detection (a reused token revokes the whole device session). Stored hashed on the server.
- **Storage on the client:** Keychain / Keystore through `flutter_secure_storage` on the app. Web uses an httpOnly, Secure, SameSite=strict cookie.
- **Devices:** each device is registered with a name, platform and public key. Users see their devices and can log one out remotely.
- **Device-bound tokens (DPoP):** each device holds a signing key in its secure chip (Secure Enclave / StrongBox, non-exportable). Every API request carries a fresh signature with that key, and the server rejects tokens presented without it. **A stolen token can't be used from any other device.**
- **Step-up:** sensitive actions (new device, consents, credit pull, export, deletion, bulk delete) need a fresh biometric or PIN check (§14.3).
- **Attestation:** sign-in and sensitive calls include a Play Integrity or App Attest verdict, so only the genuine app gets tokens.

Encryption key hierarchy

Key	Where it lives	Purpose
Vault key (existing)	Derived from the PIN (PBKDF2 600k), memory only	Unlocks the local SQLCipher database. Unchanged from v1.
User Data Key (DEK)	Created on the first device. Only ever stored wrapped.	AES-256-GCM key that encrypts every sync record and attachment.
Device key pair	X25519 per device, private key in Keychain / Keystore	Each device holds the DEK wrapped to its own public key. The server stores the wrapped copies.

Key	Where it lives	Purpose
Recovery key	Shown once (24 words), kept by the user	Derives a key that wraps the DEK for account recovery. If it's lost along with all devices, the data cannot be recovered. That's by design.
User delivery public key	Published to the server	The server seals AA and credit payloads to it, so it can write data for the user but can't read it back afterwards.
FIU key pair (server)	KMS / HSM	Required by the AA protocol to receive financial data. Never leaves KMS.

Flows

First sign-in (new user)

1. User taps Google or Apple. The app receives an ID token.
2. `POST /v1/auth/token` with the ID token and device info. The server verifies it, creates the user, identity and device, and returns access and refresh tokens.
3. The app generates the DEK and device key pair, wraps the DEK to the device key and to the recovery key, and uploads both envelopes (`POST /v1/keys/envelopes`).
4. The app shows the recovery key once and asks the user to confirm it was saved.
5. The first sync push uploads existing local data, encrypted.

Adding a second device

1. The new device signs in and registers its public key, but has no DEK yet.
2. The user scans a QR code on the new device with an existing device, or enters the recovery key.
3. The existing device wraps the DEK to the new device's public key and uploads the envelope. The new device unwraps it and pulls the sync history.

Account deletion

The user can delete their account from the app. All server rows and S3 objects are deleted within 30 days (backups expire on their own schedule), AA consents are revoked, and an audit event is kept without any financial values. Local data stays on the device unless the user also wipes it.

7 HLD: backend

Ten small .NET services behind one gateway, each with its own database, talking to each other through RabbitMQ events. Chosen deliberately so the project teaches microservices; every service stays coarse-grained so one person can build and run them.

> **Why microservices (D1).** The goal is hands-on microservice experience, not scale — at 10,000 users one API would cope. The cost is real and accepted: about 3–4 more months of work, more moving parts, and hosting of ₹5–12 L/yr instead of ₹3–8 L. Four things keep it manageable alone: **.NET Aspire** runs the whole system locally with one command; services are split by business capability, never per table; shared **building blocks** (messaging, outbox, crypto) are written once; and every service starts as one database on one shared RDS instance.

7.1 Stack

Concern	Choice	Note
Architecture	10 microservices + gateway; each its own deployable, database and pipeline	Split by capability (§7.2)
Language	C#, ASP.NET Core on .NET 10 (LTS), minimal APIs; Clean Architecture inside each service	Your strongest skill
Local orchestration	**.NET Aspire** AppHost: every service, Postgres and RabbitMQ with one command, plus a dashboard for logs, traces and metrics	Free
Gateway	**YARP** (Microsoft's reverse proxy)	Routing, TLS, JWT + DPoP validation, attestation, rate limits
Messaging	**RabbitMQ** with **Wolverine** (MIT) or **MassTransit v8** (D14)	MassTransit v9 moved to a commercial licence in 2025
Reliability	Transactional **outbox** in every publisher; **idempotent consumers** (a processed-messages table)	No event lost, none applied twice
Workflows	**Sagas** (Wolverine or MassTransit state machines) for the AA fetch and account deletion	§8.5
Data	PostgreSQL 16, one database and one login per service; EF Core + Npgsql; one RDS instance at first	No service can read another's tables
Service-to-service	Short-lived client-credentials tokens from Identity, plus mTLS inside the cluster	§14.2
Observability	OpenTelemetry traces across every hop (one correlation id per request), metrics, structured logs with PII scrubbing; Aspire dashboard locally, Grafana or CloudWatch in the cloud	A distributed bug is unfindable without traces
Admin	Blazor Server app that reads only the Audit & Reporting service	§8.10
Hosting	Docker images on AWS ECS Fargate	Simpler and cheaper than Kubernetes for one person

7.2 Services

Service	Owns (its database)	Public API (via gateway)	Publishes	Consumes
Gateway (YARP)	nothing	every public route	none	none
Identity	users, identities, passkeys, refresh tokens	`/v1/auth/*`	UserRegistered, UserSignedIn, AccountDeletionRequested	none
Devices & Keys	devices, key envelopes, pairing sessions	`/v1/devices`, `/v1/keys`	DeviceAdded, DeviceRevoked	AccountDeletionRequested
Sync	encrypted sync records, attachment refs	`/v1/sync/*`	RecordsPushed	DeviceRevoked, AccountDeletionRequested
Account Aggregator	consents, fetch sessions, sealed payloads, saga state	`/v1/aa/*`, AA webhooks, AA inbox	ConsentActivated, PayloadSealed, ConsentRevoked	AccountDeletionRequested
Credit	pull records (never report contents), sealed payloads	`/v1/credit/*`	CreditReportReady	AccountDeletionRequested
Market Data	instruments, NAV, EOD prices (public)	`/v1/market/*`	PricesPublished	none
Catalog	card products, offers, parser templates (public, signed)	`/v1/offers`, `/v1/templates`	CatalogPublished	none
Notification	push tokens, email outbox	`/v1/push/register`	none	RecordsPushed, PayloadSealed, CreditReportReady, DeviceAdded, UserSignedIn
Audit & Reporting	write-once audit, sign-in history, metrics and directory projections	internal (admin only)	none	every event
Admin (Blazor)	nothing of its own	internal	none	reads Audit & Reporting

7.3 Cross-cutting rules

- **No shared databases.** A service changes another service's data only by sending it a message or calling its API.
- **One synchronous hop at most.** Gateway → service is fine; service → service → service chains are replaced by events, so one slow service can't stall a request.
- **Outbox + idempotent consumers:** an event is written in the same transaction as the change it announces, and a consumer that sees a message id twice ignores the second copy.
- **Events are versioned** (`RecordsPushed.v1`) and change only additively; a breaking change is a new version consumed side by side.
- **Correlation id** on every request and message, so one trace follows a request through every service.
- **Idempotency keys** on mutating POSTs; **FluentValidation** at every edge, unknown fields rejected; **rate limits** at the gateway, tighter for AA and credit calls because each costs money.
- **Time and money:** UTC timestamps; money as decimal strings, never floats (same rule as v1).

8 LLD: backend

8.1 Databases, one per service

Key columns only. Every table has `created_at` and `updated_at`; `uuid` keys unless stated. **Every database** also has `outbox_messages` (id, type, payload, occurred_at, published_at) and `processed_messages` (message_id, consumer, processed_at) for reliable, idempotent messaging.

Database	Tables
`identity`	**users** (id, status, delete_requested_at, display_name_enc, email_enc, email_hash) · **identities** (user_id, provider google | apple | email, subject; unique(provider, subject)) · **passkeys** (user_id, credential_id, public_key, sign_count) · **refresh_tokens** (device_id, token_hash, family_id, expires_at, used_at, revoked_at)
`devices`	**devices** (user_id, name, platform, app_version, public_key, dpop_jkt, last_seen_at, revoked_at) · **key_envelopes** (user_id, device_id or null = recovery, wrapped_dek, alg, key_version) · **pairing_sessions** (code_hash, expires_at)
`sync`	**sync_records** (user_id, record_id, collection, seq per user, client_updated_at, deleted, ciphertext ≤ 64 KB or s3_key, key_version; PK(user_id, record_id), index(user_id, seq))
`aa`	**aa_consents** (user_id, partner, consent_handle, status, fi_types[], frequency, expires_at) · **aa_fetch_sagas** (consent_id, state, attempt, next_retry_at, error_code, cost_units) · **inbox_payloads** (user_id, sealed_payload or s3_key, expires_at, delivered_at)
`credit`	**credit_pulls** (user_id, provider, status, consent_text_version, requested_at — **no report contents**) · **inbox_payloads**
`market`	**instruments** (isin, symbol, exchange) · **prices_eod** (isin, date, close) · **mf_nav** (scheme_code, date, nav)
`catalog`	**card_products** · **offers** (product or issuer or network, merchant, category, benefit, value, cap, valid_from/to, source, verified_at) · **catalog_versions** (version, checksum, signature) · **parser_templates** (bank, channel, sender_patterns[], rule_json, version, signature)
`notification`	**push_tokens** (device_id, provider fcm | apns | webpush, token) · **email_outbox**
`reporting`	**audit_events** (actor, action, target, at, reason; exported nightly to S3 Object Lock) · **signin_events** (user_id, device_id, platform, result, ip_hash) · **admin_metrics_daily** · **user_directory** projection (id, masked email, last active, platforms, source flags) — all built from events only

8.2 API endpoints (through the gateway)

Method	Path	Service	Purpose
POST	`/v1/auth/token` · `/refresh` · `/logout`	Identity	Sign in (Google, Apple, passkey, magic link), rotate, revoke
GET · DELETE	`/v1/devices` · `/v1/devices/{id}`	Devices & Keys	List devices, log one out remotely
POST · GET	`/v1/keys/envelopes` · `/v1/devices/pairing`	Devices & Keys	Wrapped DEKs; QR pairing
POST · GET	`/v1/sync/push` · `/v1/sync/pull?since=` · `/v1/sync/attachments`	Sync	Encrypted records in and out; presigned attachment URLs
POST · GET · DELETE	`/v1/aa/consents` · `/v1/aa/fetch` · `/v1/webhooks/aa`	Account Aggregator	Consent lifecycle, fetch now, signed partner callbacks

Method	Path	Service	Purpose
POST	`/v1/credit/report`	Credit	Pull with explicit consent (one per 30 days)
GET · POST	`/v1/inbox` · `/v1/inbox/{id}/ack`	gateway fans out to AA and Credit	Fetch sealed payloads; ack deletes them
GET	`/v1/market/nav/{date}` · `/v1/market/eod/{date}`	Market Data	Whole-day price files (so holdings are never revealed)
GET	`/v1/offers/catalog?since=` · `/v1/templates?since=`	Catalog	Signed deltas (public)
POST	`/v1/push/register`	Notification	FCM, APNs or Web Push token
DELETE	`/v1/account`	Identity	Starts the account-deletion saga (24-hour cooling-off)

8.3 Events catalog

Events carry ids and timestamps only — never amounts, merchants, balances or names. Anything a consumer needs beyond that, it asks the owning service for.

Event	Publisher	Consumers	Payload
UserRegistered.v1 · UserSignedIn.v1	Identity	Reporting, Notification (new-device alert)	user_id, device_id, platform, at
AccountDeletionRequested.v1	Identity	Devices, Sync, AA, Credit, Reporting	user_id, execute_after
DeviceAdded.v1 · DeviceRevoked.v1	Devices & Keys	Sync, Notification, Reporting	user_id, device_id
RecordsPushed.v1	Sync	Notification (data-free push to other devices)	user_id, from_device_id, seq
ConsentActivated.v1 · ConsentRevoked.v1	Account Aggregator	Reporting	user_id, consent_id, status
PayloadSealed.v1	Account Aggregator	Notification	user_id, payload_id, kind
CreditReportReady.v1	Credit	Notification, Reporting	user_id, payload_id
PricesPublished.v1 · CatalogPublished.v1	Market Data · Catalog	none yet (clients poll)	date or version

8.4 Sync protocol (Sync service)

- **Record format (before encryption):** `{id, collection, updatedAt, deleted, data}`, the shape `sync_merge.dart` already merges; AES-256-GCM under the DEK with `record_id` and `collection` as associated data.
- **Push:** up to 500 records; the service stores them as-is, assigns `seq` = per-user counter + 1 and publishes `RecordsPushed`. It never merges — it can't read the data.
- **Pull:** records with `seq > cursor`; the client decrypts, runs the existing last-write-wins merge and advances its cursor.
- **Conflicts:** resolved on the client by `updatedAt`; postings are immutable once posted, so edits are new reversing and replacing postings.
- **Tombstones** kept 90 days; **key rotation** through `key_version`.

8.5 Account Aggregator fetch saga

Each consent's fetch runs as a saga in the AA service: a persisted state machine that survives restarts and knows how to undo or give up at every step.

State	What happens	On failure (compensation)
Requested	Consent request created with the partner (Finvu sandbox first); user redirected to approve	User rejects or 24 h pass: Revoked
Active	Partner webhook: consent approved; publish ConsentActivated; schedule first fetch	Consent expires or is revoked: Revoked, payloads deleted
Fetching	FI data requested for the date range	Retry with backoff, 3 attempts, then Failed and tell the user
DataReady	Partner webhook; encrypted FI data fetched and decrypted **in memory** with the FIU key (KMS)	Decrypt error: Failed, audit, no retry storm
Sealed	Normalised to Khazana DTOs, sealed to the user's delivery key, stored in `inbox_payloads`; plaintext wiped; publish PayloadSealed	none needed (nothing readable kept)
Delivered	App pulls, unseals, imports (AA entries score 1.0 once linked), acks; payload deleted	Not collected in 14 days: payload deleted

Schedule: weekly per consent, plus on app open when the last fetch is over 24 h old; skipped for users inactive 30 days. Regulatory model unchanged: a regulated partner acts as FIU, Khazana as its technology service provider (D4).

8.6 Credit report (Credit service)

1. Consent screen (bureau, purpose, validity); the consent text version is stored.
2. `POST /v1/credit/report` calls the bureau aggregator (sandbox until production); PAN and mobile are used for this call only, not kept.
3. Report sealed to the user's key and placed in the inbox; `credit_pulls` records only that a pull happened; CreditReportReady published.
4. The app stores the report in the vault; quarterly refresh by default.

8.7 Catalog: offers and parser templates

- **Offers:** curated in the admin app with sources and expiry; each catalog version is signed (Ed25519) and served as a delta. Matching against the user's cards and spending happens **on the device** — the server never sees spending. "Use the cards you have better" only, not new-card referrals (D8).
- **Parser templates:** sender patterns and rules as JSON, signed and versioned, pulled by devices daily; the device rejects unsigned or downgraded sets. Every change ships with fixture messages and must pass the shared parser suite (`test/fixtures/parser_cases.json`) in CI.

8.8 Market Data service

- **Mutual funds:** daily job downloads AMFI's public NAV file (logic from v1's `amfi_nav_provider.dart`); the client keeps a direct fallback.
- **Stocks, ETFs, bonds:** end-of-day prices from a licensed vendor once there are users; public end-of-day files for personal use while testing (free-first, §13.3).
- **Delivery:** one compressed file per day, downloaded whole by every device, so holdings are never revealed by the query.

8.9 Background jobs

Job	Service	Schedule	Notes
AA scheduled fetch	Account Aggregator	Weekly per consent, on-open if over 24 h	Starts a fetch saga; skips inactive users
Consent expiry reminder	Account Aggregator	Daily	7 days before expiry
NAV and EOD ingest	Market Data	Daily / trading days	Alert if missing by midnight
Outbox relay	every publisher	Continuous	Publishes committed events to RabbitMQ
Inbox payload expiry	AA, Credit	Hourly	Delete undelivered payloads after 14 days
Tombstone compaction	Sync	Weekly	Remove deletion markers older than 90 days
Account deletion saga	Identity (orchestrator)	After 24 h cooling-off	Every service deletes and confirms; Identity deletes last
Offers expiry	Catalog	Daily	Unpublish and republish the catalog

8.10 Admin panel

The admin panel shows **who uses Khazana and how much**: user counts, names, sign-ins, devices and system health. It can never show anyone's financial data. That rule is **enforced by the architecture, not by trust**. Financial data is end-to-end encrypted, so no server can decrypt it, and the admin side has no route to the services that hold it.

What admins can and can't see

Visible (by role)	Never visible to anyone at Khazana
Total users, new sign-ups, daily, weekly and monthly active users, retention	Transactions, balances, amounts, merchants
User's display name and masked email (r****@gmail.com), sign-up date, last active, account status	Bank accounts, card details, holdings, deposits, NPS/EPF/PPF
Sign-in history: time, platform, success or failure	Credit report, credit score
Devices: platform, model, app version, last seen	SMS, emails, notifications, receipts
Feature adoption as **yes/no flags** (AA linked, SMS capture on, Gmail linked), never values	Account Aggregator data, budgets, goals, insurance
System health: API errors, sync failures, latency, partner call counts and spend	Location, contacts, IP addresses (only salted hashes are stored)

How "can't see financial data" is enforced

1. **Encryption:** sync records and payloads are end-to-end encrypted with keys only the user's devices hold (§6). There is nothing readable to leak.
2. **Separate service, separate data:** the admin app reads only the Audit & Reporting service, whose database is filled solely from events (UserSignedIn, DeviceAdded…). Events carry no financial values (§8.3).

3. **No route to financial services:** the admin app and Reporting have no credentials or network path to the Sync, AA, Credit or Devices & Keys databases. An architecture test fails the build if one is ever added.
4. **No impersonation:** there is no "log in as user" feature, ever.
5. **No third-party analytics SDKs** (no Firebase Analytics, Mixpanel or similar). Metrics come from server events we already have (sign-ins, token refreshes, device registrations, consent status), so nothing is shared with any outside company.

Roles (least privilege)

Role	Can see / do	Typical holder
Analyst	Dashboards and aggregate counts only, with no names. Groups smaller than 5 users are hidden so no one can be singled out.	Founder, sponsor reporting
Support	User directory (name, masked email, devices, sign-in history, status). Actions: force logout, suspend, resend sign-in link. Seeing a full email needs a typed reason and is logged.	Support staff
Operations	Offers catalog, parser templates, partner cost dashboards	Ops person
Security admin	Audit log, admin access reviews, incident tools. Can't edit or delete audit records.	Security lead
Super admin	Manages admin accounts and roles. Every change needs a second super admin to approve.	2 people at most

> **While you're the only admin.** Rules that assume two people (two-person approval, separate roles) become a **24-hour delay plus an alert to your second device** for role changes, exports and key operations, so a stolen admin session can't act instantly. Your daily account uses the Analyst role, and you switch to Super admin only with a security-key tap. Split the roles properly as soon as a second person joins. The same applies to §14: "24×7 on-call" means alerts to your phone, with managed AWS services chosen to keep operations small.

Screens

Screen	Contents
Overview	Total users, sign-ups today, week and month, DAU/WAU/MAU trend, platform split (Android, iOS, web, macOS), app-version spread, retention cohorts
Users	Paginated directory: name, masked email, sign-up date, last active, platform, status. Search by exact email (matched by hash).
User detail	Identity (masked), devices, sign-in history, enabled sources as yes/no, status, support actions. No financial fields exist on this page.
Sign-in activity	Live feed of sign-ins and failures (name, platform, time), alerts for unusual patterns
System health	API error rate and latency, sync failures, job queue depth, webhook failures
Partner usage	AA fetch count, credit pulls, monthly spend against budget (totals, not per-user data)
Content	Offers catalog and parser template editors (§8.7)
Audit log	Every admin action: who, what, when, reason. Security admin only.
Admin accounts	Admins, roles, last access, quarterly access review

Security of the panel itself

- **Not on the public internet:** a separate subdomain reachable only through zero-trust access (AWS Verified Access or a VPN) plus an IP allow-list.
- **Sign-in:** company SSO with **hardware security keys (FIDO2) required**. Passwords alone are never enough. Sessions time out after 15 minutes idle and 8 hours maximum.
- **Every request** is authorised on the server against the admin's role. The UI hiding a button is never the only check.
- **Re-authentication** (security key tap) before revealing a full email, suspending a user, or changing roles.
- **No bulk export** of the user list. Any export needs two-person approval, is watermarked and logged.
- **Audit trail** of every view and action in write-once storage (S3 Object Lock), kept 180 days or more, reviewed monthly.
- **Transparency to users:** the app's Security screen shows when support accessed the user's account record and why.
- Same assurance as the product: included in every VAPT scope, strict CSP, no third-party scripts.

Data added for the panel

Table / view	Columns and notes
`users` (extend)	display_name_enc, email_enc: **field-level encrypted** with a KMS key only the admin and auth services can use. email_hash for lookup. Apple "Hide My Email" users show their relay address.
`signin_events`	user_id, device_id, at, platform, result (ok | failed | blocked), ip_hash (salted). Kept 180 days.
`admin_metrics_daily`	date, signups, dau, wau, mau, by_platform, by_version, feature-adoption counts: pre-computed aggregates
`user_directory` (projection in Reporting)	id, display name and masked email (decrypted only for the Support role), created_at, last_active_at, platforms, status, source flags
`admin_accounts`, `admin_audit`	Admins, roles, security-key ids. Append-only audit with reason codes.

Names and emails are personal data under the DPDP Act, so they stay encrypted at rest, are shown only to the Support role, and are deleted with the account.

9 LLD: automatic capture engine (device)

Every source feeds one pipeline. Confident results post to the ledger with a notification and Undo. Anything doubtful waits in the Review inbox and does not affect balances.

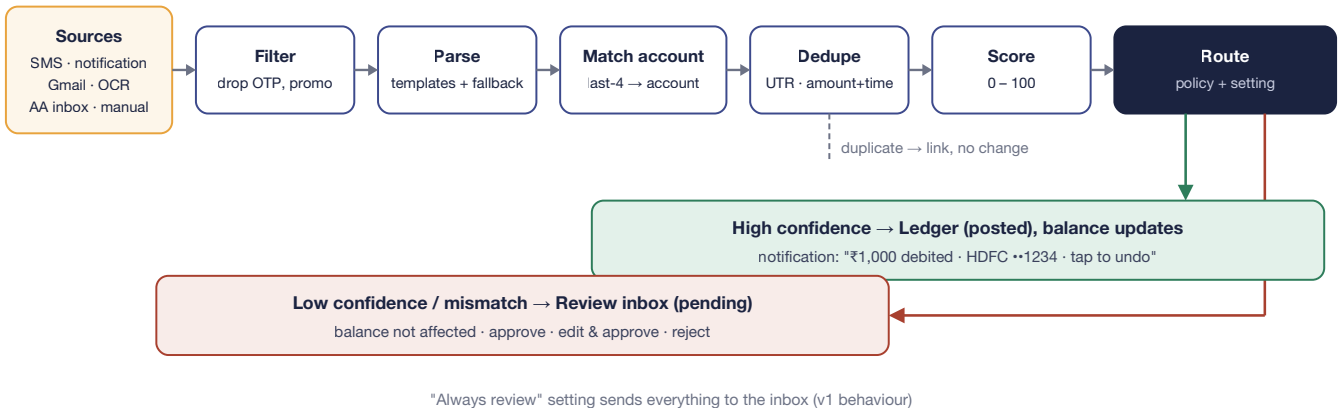


Figure 2. The capture pipeline. One path for every source. AA data arrives through the server inbox and usually scores high.

9.1 Capture channels

Channel	Platform	Implementation	Gate
SMS (live) BUILT	Android	`SmsReceiver` on `SMS_RECEIVED` (protected by `BROADCAST_SMS`), joins multipart messages, drops phone-number senders, posts to `CaptureBus`	`sideload` flavor now; Play build after its SMS declaration is approved
SMS (catch-up) BUILT	Android	On every unlock, reads the SMS inbox since the last scan (1-hour overlap; dedupe absorbs repeats) — the inbox is the durable queue, so nothing is lost while the app is closed. Opt-in 90-day import. Routed like any capture.	READ_SMS (`sideload`)
Notifications	Android	Existing `KhazanaNotificationListener`. Fallback when SMS permission is refused.	Notification access (exists)
Card emails	Android, iOS	Gmail API from the device: `messages.list` with a sender query, incremental via `historyId`, parse body, discard	Google restricted-scope verification + yearly CASA
Receipts, screenshots	Android, iOS	Share-sheet or camera → ML Kit / Vision OCR → field extraction (amount, date, merchant, UTR)	None
iPhone SMS	iOS	No API. Optional Shortcuts personal automation that forwards matching messages to an App Intent. Prototype first (unverified).	User sets it up
Bank data	All	AA payloads from the server inbox (§8.5)	Consent

9.2 Parser output (as built)

```
ParsedNotification {
  timestamp: 'YYYY-MM-DD'     // embedded date, else arrival day
  amount: Decimal             // always positive; currency token required,
                              // or SBI's "debited by 120.0" form
  type: expense | income
  kind: spend | income | refund | cardBillPayment
  instrument: account | creditCard | debitCard | upi | unknown
  merchant: string            // '' when none; e.g. "swiggy@icici"
  utr?: string                // after a keyword only — never a bare 12-digit run
  accountLast4?: string       // "XX1234" -> "1234" (3 digits when that's all)
  availableBalance?: Decimal  // never a credit-card "Avl Lmt"
  profileId?: string          // sender wins over a bank named in the body
  fingerprintInput            // 'utr|REF|direction', else
                              // 'date|amount|merchant|last4|balance'
}
```

Input is rejected above 2,000 characters and whitespace is collapsed before any pattern runs, keeping every regex linear on attacker-controlled text.

9.3 Confidence scoring (as built)

Signal	Points	If missing
Sender recognised as a bank	30	Soft reason: "Sender not recognised as a bank"
Account linked (bank + last-4 + instrument family)	25	**Forced review**: "Which account is this?"
Category learned for this merchant	15	**Forced review**: no category yet
Merchant present	10	Soft reason
Reference number present	10	none
Balance chain holds (previous alert's balance ∓ amount = this one)	10	Broken chain: **forced review**
Auto-post threshold	**80**	**Bank + account + category + merchant reach it exactly**

Also forced, whatever the score: refund, card-bill payment, the other leg of a transfer (same reference, opposite direction), an amount over 3× the account's 90-day 95th percentile (with at least 5 entries of history), a hand-typed entry of the same amount within ±2 days, and the "Always review" switch. A forced reason always wins over the score. The threshold is 80, not the 85 first planned: at 85 a normal card SMS without a reference number could never post itself.

9.4 Capture lifecycle

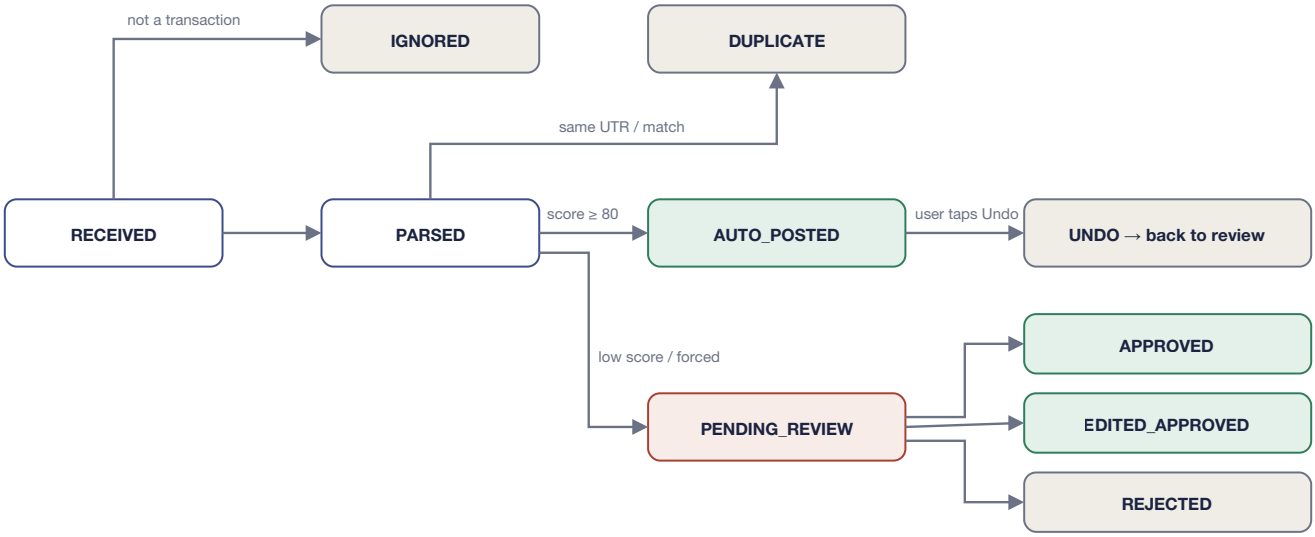


Figure 3. Lifecycle of one captured message (as built). Only AUTO_POSTED, APPROVED and EDITED_APPROVED have an entry in the ledger. Undo removes the entry and returns the capture to PENDING_REVIEW with the reason "You undid this"; a rejected capture can be restored the same way. Decided captures are kept 90 days as the dedupe record.

9.5 Ledger rules

- **Balance** = opening balance + Σ posted postings. Pending captures appear separately as "₹X waiting for review".
- **Edit and approve** stores `parsed_amount` and `final_amount`. The difference teaches the template (see 9.7).
- **Undo** removes the entry and its postings through `TransactionNotifier.remove` and returns the capture to the inbox; the capture row keeps the history. Approve is idempotent, so a double tap writes one entry.
- **Transfers between own accounts** (a debit SMS from account A and a credit from account B with the same amount within 10 minutes) are offered as a single transfer, not as expense plus income.
- **Card bill payments** (a bank debit to a card) are a liability payment, not spending, so totals aren't counted twice.

9.6 Cross-source dedupe

1. Same **reference and same direction** → the same transaction. The key includes the direction so both legs of a transfer between own accounts survive — and are flagged as a transfer.
2. A **hand-typed entry** with the same amount and direction within ±2 days → flagged as a likely duplicate and sent to review (entries auto-capture wrote don't count).
3. The **fingerprint** blocks repeats in any status: a rejected capture is still the record that this alert was handled.
4. *Phase 4:* source priority when merging with AA data: **AA > SMS > email > notification > OCR**.

9.7 Balance reconciliation and learning

- **Built:** each alert's "Avl Bal" must follow from the previous alert's for the same account. It deliberately compares bank figures with bank figures — most people never enter an exact opening balance, so a check against Khazana's computed balance would flag everything. A break sends the capture to review. *Planned:* a "possibly missing transaction" banner with Find missing (AA fetch, Phase 4).
- *Planned:* repeated user edits for the same template and field raise a local "template suspect" flag. With opt-in, an anonymous signal (template id and field only, no message text) helps us fix templates centrally.
- Merchant aliases and categories learned from user choices reuse the existing `merchant_alias` table.

9.8 Capture setup by platform

The app detects the platform and shows only the setup that works there. Android gets a standard permission flow. iPhone gets a guided Shortcuts setup in which every step is one tap. The web app explains that SMS capture needs the mobile app.

Platform	What the user sees	Notes
Android	1. Card: "Track bank SMS automatically". A plain explanation: read on this phone, only amount, date, merchant and account are kept. 2. Tap "Turn on", which shows the system SMS permission. 3. Optional: "Also capture from bank-app notifications", which opens notification access. 4. Optional: "Import the last 90 days", which sends the results to the Review inbox.	Google Play requires the explanation screen (prominent disclosure) before the permission prompt. If SMS is refused, notification capture is offered as the fallback.
iPhone	1. Card: "Set up automatic SMS tracking (about 1 minute)". 2. **Add shortcut:** one tap opens our ready-made "Send to Khazana" shortcut, and the user taps Add. 3. **Create automation:** one tap opens the Shortcuts app, next to a 4-step picture guide (Automation → Message → sender contains your bank → Run Immediately) and a 30-second video. The app lists the bank sender names to use, taken from the user's accounts. 4. **Test it:** the app waits for the next bank message, or asks the user to forward one, and shows "Working" with a green tick.	Apple doesn't let apps create automations, so the user does step 3. The shortcut runs our App Intent in the background without opening the app. If the user skips this, AA, Gmail and screenshot sharing still work.
Web (Safari, Chrome)	Banner: "Automatic SMS tracking needs the Khazana mobile app", with App Store and Play links. Offers Gmail linking and screenshot import instead.	Browsers can't read SMS.
macOS / desktop	No SMS setup shown. Data arrives from the phone through sync.	n/a

Keeping it working

- **Connections screen** shows each source per device: On, Off, Needs setup, and "last captured 2 h ago".
- **Health check:** if a source that used to capture has been silent for 14 days while bank activity shows up elsewhere (AA or email), the app nudges: "Your SMS shortcut may have stopped. Tap to check."
- **New bank detected** (for example a new AA-linked account): on iPhone, prompt to add that bank's sender to the automation.
- **Implementation:** `Platform.isAndroid` / `Platform.isIOS` in Flutter decide which card shows. On web, a user-agent check shows the banner only. The setup state lives in the device database (not synced, because it's per device).

10 Data storage

Readable financial data lives only on devices. The server keeps account metadata, consents, encrypted sync records and public catalogs.

10.1 Storage map

Store	Technology	Holds	Encryption
App database	Drift on SQLCipher	Full ledger, accounts, cards, holdings, captures, credit report, offers cache	AES-256, vault key from PIN
Web database	Dexie (IndexedDB)	Same model as encrypted records	AES-256-GCM per record
Device secrets	Keychain / Keystore	Salt, device private key, refresh token	OS hardware-backed
Server databases	PostgreSQL 16, one per service	Users, devices, consents, sync records (ciphertext), catalogs, audit	RDS encryption at rest plus end-to-end for sync data
Blob storage	S3	Encrypted attachments, large sync records, sealed payloads over 64 KB	End-to-end plus SSE-KMS
Messaging and cache	RabbitMQ; Redis later	Events between services (ids and timestamps only); cache and rate limits when needed	TLS in transit, no financial data
Secrets	KMS, Secrets Manager	FIU private key, partner API keys, signing keys	HSM-backed

10.2 Device schema (Drift v7, as built)

Additive and idempotent like the rest of the ladder: every v7 column is guarded, because v3 creates `pending_captures` from the current definition, and the v3 step adds the new transaction columns before its generated select runs.

Table	Status	Columns
`pending_captures`	extended (v7)	status (pending | auto_posted | approved | edited_approved | rejected | reversed), confidence 0–100, reason_codes, account_id, category_id, utr, account_last4, available_balance, instrument, kind, profile_id, final_amount, decided_at, posted_txn_id · indexes (vault_id, fingerprint) and (vault_id, status, captured_at)
`transactions`	extended (v7)	source (null = manual, sms, notification), capture_id
`capture_account_links`	new (v7)	id, vault_id, profile_id ('' = no bank named), last4, instrument, account_id, created_at · unique (vault, profile, last4, instrument)
`cards`, `credit_reports`, `retirement_accounts`, `deposits`, `offers_cache`, `sync_state`, `parser_templates_cache`	planned	Added by the phase that needs them (Cards in 1.7, sync in P2, AA data in P4, offers in P5), each in its own migration step

10.3 Retention

Data	Where	Kept for
Raw SMS, notification and email text	Device memory	Discarded right after parsing (existing rule)

Data	Where	Kept for
AA and credit plaintext	Server memory	Seconds, while processing
Sealed inbox payloads	Server	Until acknowledged, at most 14 days
Sync records	Server	While the account exists. Tombstones 90 days.
Logs	Server	180 days, stored in India (CERT-In), scrubbed of personal data
Audit events	Server	As required by partner agreements and law. No financial values.
Deleted account data	Server	Purged within 30 days. Backups age out within 35 days.

11 Module-by-module implementation

Fourteen work modules, about 30–42 person-months in total; M3 is built, leaving about 27–38. Effort is for an experienced developer — learning adds time, AI pair-programming saves it. §15 turns this into dates.

Module	Client work	Server work	External	Effort
M0 Microservice platform	None	Aspire AppHost and ServiceDefaults, YARP gateway, RabbitMQ, building blocks (messaging, outbox, idempotency, crypto), per-service CI, tracing	None (local first)	3–4
M1 Login and keys	Sign-in screens, secure token storage, DPoP keys, key generation, QR pairing, recovery key	Identity and Devices & Keys services, token rotation, passkeys, envelopes	Google, Apple sign-in, SES	3–4
M2 Sync	Push/pull engine over the existing merge, attachments, sync status UI (Flutter and web)	Sync service, seq allocator, presigned URLs, compaction	S3	2–3
M3 Capture engine BUILT	Parser v2, router, pipeline, review inbox, Undo, SMS in the sideload flavor (PR #28)	Template signing moves to Catalog (M7)	Gmail, ML Kit later (P3)	3–4
M4 Accounts and AA	Link-account flow, consents, accounts hub, inbox import	AA service: consents, webhooks, fetch saga, normalisers, sealing	Finvu sandbox, then a regulated partner	3–4
M5 Cards	My cards, statement and due dates, utilisation (Slice 1.7, prototype first)	None beyond sync	None	1–1.5
M6 Credit report	Consent, viewer, refresh rules	Credit service, aggregator client, sealing	Bureau aggregator sandbox	1–1.5
M7 Offers and templates	Catalog sync, on-device matcher, savings insights	Catalog service, signing, admin editors	Manual curation	2–3
M8 Investments	AA holdings, FD/RD, NPS, EPF/EPS/PPF, passbook PDF import	Market Data service: NAV and EOD ingest, price files	AMFI, EOD source	2–3
M9 Insurance	Policies from AA, policy PDF upload	Insurance normaliser in the AA service	AA	0.5–1
M10 UI and motion	Prototype, motion system, navigation, refresh of key screens	None	None	2.5–3
M11 Admin and operations	Blazor admin: dashboards, directory, sign-in feed, editors, audit	Audit & Reporting service, projections, RBAC, zero-trust access	SSO + FIDO2	2–2.5
M12 Quality and compliance	Fixture suites, integration and E2E tests, accessibility	Contract tests between services, load tests, DPDP processes	Play and Google verifications	2–3

Module	Client work	Server work	External	Effort
M13 Security hardening	DPoP signing, step-up, attestation, root detection, screen protection, pinning	DPoP verification at the gateway, risk engine, mTLS, SIEM, runbooks, HSM ceremonies	CERT-In auditor, crypto reviewer	3–4
Total (M3 built: about 27–38 remaining)				**30–42**

Definition of done (every module)

- Domain logic in both Dart and TypeScript where both clients need it, with mirrored tests and shared fixtures (the parity rule).
- **Per service:** its own tests and container build in CI, an OpenAPI spec, contract tests for every event it publishes or consumes, outbox for publishing, idempotent consumers, traces carrying the correlation id.
- No financial values in logs, events, analytics or crash reports, checked by a test.
- Works offline where the section says it should; online-only actions queue.
- Design sign-off against the prototype, in light and dark themes.

12 External APIs and their cost

AA fees and credit report pulls are the two costs that grow with users. Everything else is cheap or free. Verify every figure with the vendor; sources are in Appendix B.

Service	Used for	Provider options	Cost (estimate)	Notes
Account Aggregator	Bank accounts, FD/RD, MF, stocks, ETF, NPS, insurance	Setu, Finvu, OneMoney, Anumati (through a regulated FIU partner)	₹0.30–₹9.90 per account per fetch (older OneMoney rate card). Data API tiers ₹1–₹60 per account.	Needs a regulated FIU, sandbox testing and certification. EPF and PPF not yet live on AA.
Credit bureau	Credit score and report	Through aggregators (Surepass, Decentro, Signzy, Perfios) or direct	About ₹5–₹50 per pull (third-party range, unverified)	Eligibility criteria apply. Quarterly refresh by default.
Gmail API	Credit card alert emails	Google	API free. Yearly security assessment (CASA) for restricted scopes: roughly ₹0.5–4 L/yr	OAuth verification takes weeks. Plan early.
On-device OCR	Receipts, screenshots	Google ML Kit, Apple Vision	Free	Runs offline
Mutual fund NAV	MF valuations	AMFI daily file	Free	Already used in v1
Stock / ETF prices	EOD valuations	Licensed vendor (Global Datafeeds, TrueData and others) or exchange licence	Vendor feeds from about ₹27,000/yr per segment. A redistribution licence costs more: budget ₹1–5 L+/yr.	End-of-day only. Real-time is far more expensive.
Sign-in	Google, Apple	Google Identity, Sign in with Apple	Free	Apple Developer Program $99/yr. Play one-time $25.
Push	"New data" pings	FCM, APNs	Free	No amounts in payloads
Email	Magic links, alerts	Amazon SES	About $0.10 per 1,000 emails	Negligible
Error tracking	Crashes, errors	Sentry (SaaS) or GlitchTip (self-hosted)	Free to about ₹30k/yr	Scrubbing mandatory
Card offers data	Offers catalog	Manual curation from bank and network sites	Staff time (see §13)	No reliable public API

Why fetch frequency matters (AA)

10,000 users × 3 accounts	At ₹0.30 / account-fetch	At ₹1.00 / account-fetch
Daily fetch	≈ ₹33 L / yr	≈ ₹1.1 Cr / yr
Weekly fetch (recommended, plus SMS and email for real-time)	≈ ₹4.7 L / yr	≈ ₹15.6 L / yr
Weekly + on-open when stale, skipping inactive users	≈ ₹3–6 L / yr	≈ ₹10–20 L / yr

SMS and email give real-time updates at no per-use cost, so AA's job is completeness and catching missed entries, not speed. That design choice cuts AA cost by about 80%.

13 Cost: what you will actually spend

Building alone, your time isn't a cash cost: about ₹8–28 lakh goes out before launch, mostly on security assurance. Everything runs locally on free tools while you build; paid services switch on one by one only when testing proves the need.

13.1 Before the v2 launch (Oct 2026 – Dec 2028)

Item	Cost	When
Local development (Aspire, Docker, Postgres, RabbitMQ, MinIO)	₹0	Throughout
AWS staging for the microservices	₹0.5–2 L / yr	From late P2. New-account credits ($100–200, 6 months on the Free plan) cover the start — open the account only when staging is needed.
AI coding assistant subscription	₹0.2–0.5 L / yr	Throughout
Domain, email, certificates	≈ ₹0.05 L / yr	P1
Google Play developer account	$25 once	If not already registered
Apple Developer Program	$99 / yr	P3 (iOS)
Test devices (mid-range Android, older iPhone)	₹0.3–0.6 L once	P1, P3
Company registration + accountant	₹0.3–1 L + ₹0.2–0.5 L / yr	Before partners and paid features
Legal drafting: privacy policy, terms, consent texts	₹0.5–2 L	Before other people's data is held
Independent cryptography and architecture review	₹2–6 L	Before the P2 release
CERT-In-empanelled VAPT (web, API, mobile)	₹2–8 L	Before the P2 release; re-test before launch
Google OAuth verification + first CASA	₹0.5–4 L	When Gmail goes past 100 test users (self-scan is free)
AA partner onboarding and certification	Quote needed	Before production bank data
Total before launch	**≈ ₹8–28 L**	**Mostly security assurance in P2–P3**

13.2 After launch: yearly cash costs

Item	~1,000 users	~10,000 users	Driver
Cloud: 10 services × 2 small Fargate tasks, gateway, RDS, broker, S3, CDN, monitoring, backups	₹4–8 L	₹5–12 L	Mostly fixed (one task per service runs all the time)
Account Aggregator fees	₹0.5–3 L	₹5–33 L	Fetch frequency (§12)
Credit report pulls	₹0.6–6 L	₹6–60 L	Price per pull × refresh rate
Stock price data licence	₹1–5 L	₹1–5 L	Needed once others see the prices
Google CASA (yearly)	₹0.5–4 L	₹0.5–4 L	Assessment tier
CERT-In VAPT (yearly)	₹2–8 L	₹2–8 L	Scope

Item	~1,000 users	~10,000 users	Driver
Bug bounty	₹0–5 L	₹2–10 L	Rewards paid
Email, push, store fees, error tracking, security monitoring	₹1–3 L	₹1.5–5 L	Small
Legal, accountant, compliance	₹1–3 L	₹1–3 L	Fixed
Commercial app shielding (optional, D11)	₹0	₹0–8 L	Only if the pentest needs it
Cash total per year	**≈ ₹11–45 L**	**≈ ₹24 L–1.5 Cr**	**Fixed costs dominate at 1,000 users**

Help to hire when revenue allows

Role	Per year	When it becomes necessary
Maintenance developer	₹12–30 L	Around 10,000 users, or when templates break faster than you can fix them
Security engineer (part-time or consultant)	₹6–15 L	Before production AA — partners will ask who owns security
Support person	₹2–6 L	When support takes more than 5 hours a week
Offers curation	₹2–4 L	P5 onwards

13.3 Free first, paid later (adapters)

Every paid dependency sits behind an interface chosen by configuration (`Providers:*`), so moving from the free option to the paid one is a new adapter plus one config line — no other code changes. A startup guard refuses any Mock adapter in Production, and one contract-test suite runs against every adapter.

Capability	Interface	Free now	Paid later
Bank data	`IBankDataProvider`	Sample data, then the **Finvu sandbox** (free; token renews every 6 months)	Finvu or Setu production through a regulated partner (quoted per fetch)
Credit report	`ICreditReportProvider`	Sample report / aggregator sandbox	Bureau aggregator (per pull)
Prices	`IPriceProvider`	AMFI NAV (free) + public end-of-day files for personal use	Licensed EOD vendor
Card emails	Gmail reader	Google "testing mode" (≤ 100 test users, no verification)	Verified app + yearly CASA
Email	`IEmailSender`	Console / free tier	Amazon SES
Files and keys	`IBlobStore`, `IKeyVault`	MinIO, local dev keys	S3, KMS
Database, broker	connection strings	Docker locally; Neon free tier, CloudAMQP "Little Lemur" (1 M messages/month) for a friend's test	RDS, Amazon MQ
Monitoring	OpenTelemetry	Aspire dashboard; Grafana Cloud free (10k series, 50 GB logs); Sentry free (5k errors)	Paid tiers

Free-tier limits change — two of these changed during 2026 alone. Keep the current numbers in `docs/free-tier-limits.md`, and put a budget alarm on every account that has a card attached. An `IEntitlements` hook (all features on) is built from the start in case a premium plan is chosen (D13).

Cost levers

- **Fetch AA weekly** and rely on SMS and email for real-time — the biggest saving.
- **Refresh credit reports quarterly**; skip AA fetches for users inactive 30 days.
- **End-of-day prices only.**
- **Smallest Fargate size per service**, one task each until load says otherwise; services with no traffic overnight can scale to zero.
- **Switch on paid partner features only when revenue covers them.**

> **Monetisation (open decision D13).** Options that keep today's free features free: a B2B version; a paid online tier covering server and partner costs, with the offline app staying fully free; or a sponsor. Decide before AA and credit reports go live in production.

14 Security architecture and compliance

Khazana v2 handles real bank data, so security is a release blocker, never a trade-off. The baseline is defence in depth: no single failure (a stolen token, a breached server, a lost phone or a rogue insider) can expose a user's financial data.

14.1 Standards we build and test against

Standard	Applies to	Target
OWASP MASVS (mobile app security)	Flutter apps	**Level 2 + Resilience**, the level used for banking apps, verified by an external mobile pentest
OWASP ASVS (application security)	Backend, API, admin	**Level 3**, the highest, for critical applications
OWASP API Security Top 10	All APIs	Every item covered by a test or control
NIST SP 800-63B (authentication)	Login and sessions	Phishing-resistant authenticators (passkeys) and device-bound sessions
CERT-In Directions (2022)	Whole service	Report incidents within 6 hours. Keep logs for 180 days, stored in India.
DPDP Act 2023	Personal data	Consent, safeguards, breach notification, deletion on request
ISO/IEC 27001	Organisation	Certification within 12–18 months of launch. Partners and B2B customers will ask for it.
PCI DSS	Card data	**Out of scope by design:** we never touch card numbers or CVV (see 14.2)

> **Never stored, anywhere, ever:** full card numbers, CVV, card expiry, UPI PIN or MPIN, net-banking usernames or passwords, OTPs, raw SMS or email text, Account Aggregator plaintext, and PAN (tax ID) beyond the single credit-report request. Masked last-4 digits are the most we keep. A CI test fails the build if any of these patterns appears in logs, analytics or crash reports.

14.2 Defence in depth

Layer	Controls
Identity	**Passkeys** as the preferred sign-in, plus Google or Apple. No passwords, no SMS OTP (immune to SIM swap). **Device-bound tokens (DPoP)**: every request is signed by a private key in the phone's secure chip (Secure Enclave / StrongBox, non-exportable), so a stolen token is useless on any other device. 15-minute access tokens, rotating refresh tokens with reuse detection, step-up biometrics for sensitive actions (14.3), new-device alerts, login history, and "log out everywhere". Risk signals (new country, impossible travel, failed attestation) trigger step-up or a block.
App integrity	Google Play Integrity and Apple App Attest checked at sign-in and on every sensitive call, so only the genuine, unmodified app is served. Root, jailbreak, hooking and debugger detection disables sensitive features. Flutter obfuscation with separate debug symbols. Signature and tamper checks. Commercial app shielding (RASP) if the pentest finds gaps (D11).

Layer	Controls
Data on the device	SQLCipher AES-256 database (existing). Keys in the hardware keystore. PIN never stored. PBKDF2 600k for the vault (existing) and Argon2id for the new recovery key. Auto-lock (existing). Screen-capture blocking on sensitive screens (FLAG_SECURE on Android, a protected view on iOS). App-switcher blur. **Lock-screen notifications hide amounts by default** ("New transaction to review"). Clipboard auto-clear. Optional wipe after 10 wrong PINs.
Cryptography	Vetted libraries only (libsodium, platform crypto): no home-made cryptography. AES-256-GCM, X25519, Ed25519, HKDF-SHA-256, Argon2id. `key_version` on everything so keys can rotate. **Hybrid post-quantum key exchange** (X25519 + ML-KEM-768) planned for long-lived sync keys once library support is stable on all platforms, because financial history must stay private for decades. An independent cryptography review of the key hierarchy before launch.
Network	TLS 1.3 only, HSTS preload. **Certificate pinning** with two backup pins and a remote kill-switch. mTLS plus IP allow-lists for partner connections. Signed webhooks with a replay window.
Cloud	Separate AWS accounts per environment, with organisation-wide guardrail policies. Private subnets, no public database. WAF and DDoS protection. Least-privilege IAM, no long-lived access keys. **FIU and signing keys in FIPS-validated HSMs** (KMS; dedicated CloudHSM only if a partner requires it). Encryption at rest everywhere. Signed container images. Immutable infrastructure. Audit logs in write-once storage (S3 Object Lock).
Between services	Every service-to-service call carries a short-lived client-credentials token from Identity, scoped to that call, plus mTLS inside the cluster. RabbitMQ over TLS with one user per service, allowed to publish and consume only its own exchanges and queues. One database login per service, with rights on its own database only. Events carry ids, never financial values. Audit & Reporting has no route to the Sync, AA, Credit or Devices & Keys databases (§8.10).
People and access	No standing production access: just-in-time access with approval and session recording. **Hardware security keys (FIDO2) for every admin.** Two-person approval for production deploys and key operations. Background checks for staff with infrastructure access. By design, staff can't read user financial data (end-to-end encryption).
Monitoring and response	CloudTrail, GuardDuty and Security Hub feeding a central SIEM. Alerts on sign-in anomalies, mass deletes, partner-cost spikes and unusual key use. 24×7 on-call. Written incident-response plan: CERT-In report within 6 hours, DPDP breach notice to the Board and affected users. Scrubbed logs kept 180 days in India. Quarterly tabletop drills.
Secure development	Threat model for every feature (extending `docs/THREAT–MODEL.md`). SAST (CodeQL, Semgrep), dependency scanning with an SBOM, secret scanning, DAST against staging. Mandatory review with a security checklist. Signed commits and release builds. Pinned dependencies.
Assurance	**CERT-In-empanelled VAPT** (web, API, mobile) before every phase that adds server surface, and yearly after that. Mobile pentest against MASVS L2+R. Independent cryptography review. Private bug bounty from launch, public later. ISO 27001 certification.
Resilience	Encrypted backups, point-in-time recovery, and a disaster-recovery copy in AWS Hyderabad (ap-south-2), so data stays in India. Recovery point 15 minutes or less, recovery time 4 hours or less. Restore drills every quarter.

14.3 Sensitive actions need step-up

A valid session isn't enough for these. Each one needs a fresh biometric or PIN check on the device, signed with the device key.

Action	Extra protection
Sign in on a new device	Approval from an existing device or the recovery key, plus push and email alerts to every device

Action	Extra protection
Add or remove a device, "log out everywhere"	Biometric or PIN on a trusted device
Link or revoke a bank (AA) consent	Biometric or PIN, plus the AA's own OTP
Pull a credit report	Biometric or PIN, plus explicit consent screen
Export all data, change the recovery key	Biometric or PIN, plus an alert to the other devices
Delete the account	Biometric or PIN, then a 24-hour cooling-off period that can be cancelled from any device
Bulk delete (more than 50 records)	Biometric or PIN. The server keeps previous versions for 30 days so it can be undone.
Turn off a security feature (for example, showing amounts on the lock screen)	Biometric or PIN

14.4 What happens if something is stolen

If an attacker gets…	Result
An access token	**Useless.** Requests must be signed by the device's secure-chip key, which can't be copied. Expires in 15 minutes anyway.
A refresh token	**Useless** for the same reason. Reuse detection also logs out that device's session.
A locked phone	Data is encrypted under the vault PIN and keys sit in secure hardware. The user logs the phone out remotely from another device.
An unlocked phone	The app still needs PIN or biometric. Sensitive actions need step-up. The device can be logged out remotely.
Our whole server database	Only ciphertext and metadata. No AA or credit plaintext is ever stored, and HSM-held keys can't be exported.
An insider with cloud access	Same as above, plus just-in-time access, two-person approval and recorded, write-once audit logs.
The user's password (phishing)	There are no passwords. Passkeys can't be phished.
The user's SIM (SIM swap)	Nothing. SMS OTP is never used for sign-in or recovery.
A tampered parser template	Rejected. Templates are Ed25519-signed and version-pinned.
A fake bank SMS	Sender allow-list, balance-consistency check and unusual-amount rule send it to the Review inbox, not the ledger.

14.5 Approvals and certifications

Item	Owner	What's needed
Google Play SMS permissions	Mobile lead	Permissions Declaration Form under the SMS-based money management exception, a demo video and a prominent disclosure screen. Approval isn't guaranteed. Notification capture is the fallback.
Google restricted scope (Gmail)	Mobile lead + security	OAuth app verification, privacy policy, limited-use declaration, yearly CASA security assessment

Item	Owner	What's needed
Apple App Store	Mobile lead	Sign in with Apple, privacy nutrition labels, account deletion in the app, App Attest
Account Aggregator	Backend lead + business	Regulated FIU partner agreement, sandbox integration, certification by a Sahamati-empanelled certifier, and the partner's own security audit
Credit bureau	Business	Aggregator agreement, eligibility checks, consent records
CERT-In VAPT certificate	Security lead	From an empanelled auditor, before launch and yearly. Regulated partners usually ask for it.
DPDP Act	Project lead	Notice and consent, purpose limitation, security safeguards, breach process, deletion on request, grievance contact

14.6 Privacy commitments (user-facing)

- Messages and emails are read on the phone. Only the amount, date, merchant and account are kept.
- Your synced data is locked with your key, and Khazana's servers and staff cannot read it.
- We never ask for or store your card number, CVV, UPI PIN or bank password.
- Every source can be switched off in Connections, and every consent can be revoked.
- A Security screen shows your devices, sign-in history and alerts, with "log out everywhere".
- No ads, no selling data, no loan offers.

15 Solo build plan: roadmap, learning path and folder structure

About 27 months alone, learning as you go. Phase 1 is already built; the rest front-loads learning, ships something real every few months, and starts paperwork early so it never blocks the code.

> **Assumptions.** About 25–30 hours a week alongside your job, with AI pair-programming for code, reviews and tests. Microservices add about 3–4 months over a single backend — accepted, because learning them is a goal. At about 15 hours a week, plan for roughly 40 months, or move P5 to a later v2.1. Every phase ends with something shipped, so the project stays useful and resume-worthy if the schedule slips.

15.1 Roadmap

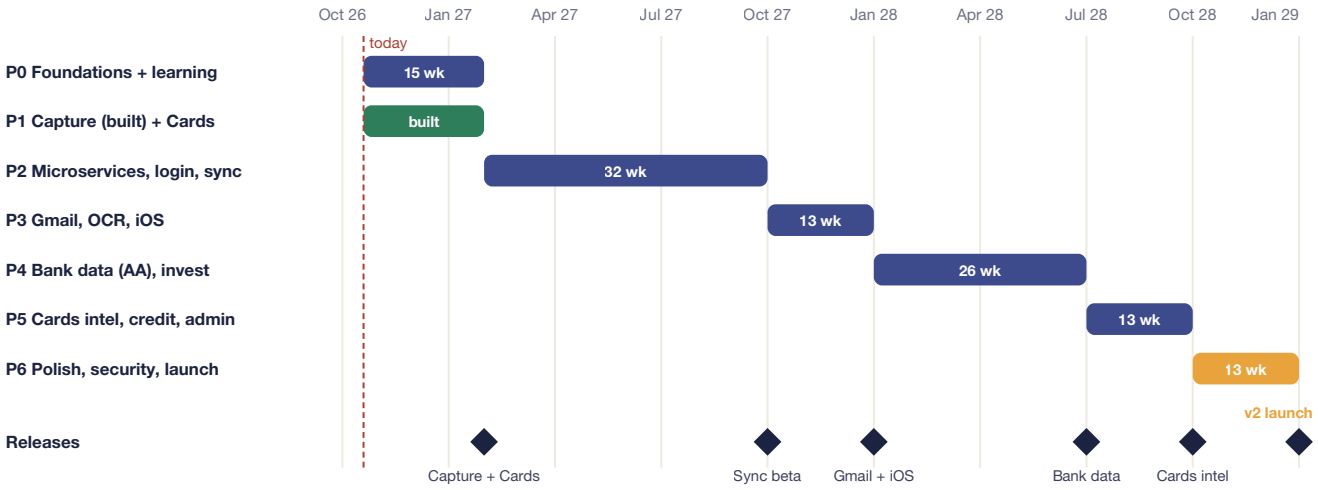


Figure 4. Phase timeline, 19 October 2026 to December 2028. Green is Phase 1 (core built in October 2026, PR #28); amber is the v2 launch.

15.2 Phases in detail

Phase and dates	What you build	What you learn	Ships, and the resume line
P0 Foundations Oct 2026 – Jan 2027	Dart and Flutter basics through the Phase 1 code; ADRs for D1–D15 in `docs/adr/`; `backend/`, `contracts/`, `infra/` folders; an Aspire "hello" with two services and RabbitMQ; clickable prototype of Accounts, Cards and Connections.	Dart, Flutter, Riverpod, Drift; Aspire, Docker, RabbitMQ basics	Internal. "Learned Flutter by extending a production app with 237 Dart files and 600+ tests."
P1 Android capture core built Oct 2026; finish by Jan 2027	**Built (PR #28):** parser v2, Drift v7, router, pipeline, review inbox, Undo, SMS in the sideload flavor. **Left:** test on a real phone and merge; Slice 1.7 Cards (prototype first); PRD §13 update; Play SMS declaration only when publishing SMS on Play.	Kotlin basics, Android permissions, Play policy	Tester build. "Built an on-device engine that parses 25+ Indian bank SMS formats and auto-posts with confidence scoring, with a ReDoS fix."

Phase and dates	What you build	What you learn	Ships, and the resume line
P2 Microservices, login, sync Feb – Sep 2027	In order: platform skeleton (Aspire, YARP, RabbitMQ, outbox) + **Market Data** as the first service (4 wk) · **Identity** with passkeys and DPoP (4) · **Notification**, the first pure consumer (1) · **Devices & Keys** (2) · **Sync** (4) · **Audit & Reporting** + Blazor admin v1 (3) · **Catalog** (2) · Flutter and web sync clients (4) · Terraform and per-service CI (4) · crypto review and VAPT (4).	YARP, Wolverine or MassTransit, outbox and idempotency, OAuth/ OIDC, DPoP, WebAuthn, libsodium, AWS, Terraform	Multi-device beta. "Built a .NET microservices platform: YARP gateway, RabbitMQ messaging with outbox, end-to-end encrypted sync, device-bound tokens."
P3 Gmail, OCR, iOS Oct – Dec 2027	Gmail reader on device (testing mode first), ML Kit / Vision OCR, share-sheet import, iOS App Intent with the Shortcuts guide.	Gmail API, ML Kit, Swift and App Intents	Android and iOS update. "Shipped privacy-first email and receipt capture with on-device OCR."
P4 Bank data and investments Jan – Jun 2028	**Account Aggregator** service with the fetch saga against the Finvu sandbox (5 wk); accounts hub; holdings, FD/RD, NPS, insurance from AA; EPF/EPS/PPF manual and passbook PDF; production partner talks.	AA (ReBIT/Sahamati) specs, sagas, webhook security, KMS	Automatic bank data. "Integrated India's Account Aggregator framework with a saga-orchestrated fetch."
P5 Cards intelligence Jul – Sep 2028	**Credit** service (2 wk, sandbox first); offers catalog with the on-device matcher; full admin roles.	Blazor, RBAC	Cards section complete. "Built a privacy-preserving card-savings engine."
P6 Polish and launch Oct – Dec 2028	Motion refresh, accessibility, performance; full VAPT re-test and MASVS pentest; bug bounty; store listings; launch.	MASVS testing, Flutter DevTools	v2 public launch and an updated case study.

15.3 Paperwork track (start early)

Item	Start	Needed by
Company registration (Pvt Ltd or LLP)	Feb 2027	Organisation store accounts and partners
Privacy policy, terms, consent texts	Mar 2027	Before friends' real data is held
Book the crypto review and VAPT	Jun 2027	P2 release (Sep 2027)
Google OAuth verification + CASA	Aug 2027	Gmail beyond 100 test users
Finvu sandbox token	Nov 2027 (renew every 6 months)	P4 start (Jan 2028)
Monetisation decision (D13)	By Dec 2027	Before paid partner features
Regulated AA partner (Finvu or Setu production)	Jan 2028	Production bank data
Credit bureau aggregator	Apr 2028	P5 (Jul 2028)
Play SMS permission declaration	When publishing SMS on Play	A Play build with SMS

15.4 Learning path

Skill	When	Why you need it	Where to learn
Dart and Flutter	P0	Every screen; reading Phase 1	dart.dev tour, docs.flutter.dev codelabs; then read `lib/features/capture`
Riverpod, Drift	P0	State and the device database	riverpod.dev, Drift docs; trace `capture_providers.dart`
Docker and .NET Aspire	P0	Running ten services locally	Microsoft Learn (Aspire), Docker docs
RabbitMQ + Wolverine or MassTransit	P0–P2	Events, outbox, idempotent consumers, sagas	RabbitMQ tutorials; the library's docs
YARP	P2	The gateway	Microsoft docs
OAuth 2 / OIDC, JWT, DPoP, WebAuthn	P2	Login and sessions	Microsoft identity docs, RFC 9449, webauthn.guide, Fido2NetLib
Applied cryptography	P2	End-to-end sync, sealing	libsodium and NSec docs — never write your own primitives
AWS + Terraform	P2	Hosting as code	AWS Skill Builder (free), HashiCorp tutorials
Kotlin, Swift + App Intents	P1, P3	Native capture	kotlinlang.org; developer.apple.com
OWASP ASVS and MASVS	P2, P6	The security baseline (§14)	owasp.org
Blazor Server	P2, P5	Admin app	Microsoft docs

15.5 How you work each week

- **Time:** weekday evenings of 2–2.5 hours, 6–8 hours across the weekend; roughly 70% building, 20% learning, 10% writing.
- **Every task:** GitHub issue → branch in a git worktree → tests first for domain logic → code → self-review plus AI review → PR into `feature/safety-net-and-hardening` (the trunk accepts PRs only) → CI green → merge.
- **Every month:** a tester build; update the case study, resume bullets and this roadmap.
- **Every phase end:** a short retro, the §14 security checklist, an update to this document.

You wear every hat

Hat	How you cover it
Architect	This document plus ADRs in `docs/adr/`
Developer	You, with AI pair-programming
QA	Automated tests, the shared parser fixtures, contract tests, 10–20 friendly testers
Designer	The existing Zentra kit, reference apps for motion, a clickable prototype before each new section
Security	The §14 baseline plus paid external checks
Operations	Managed AWS services, traces in one dashboard, alerts to your phone, a status page
Buy, don't build	VAPT, cryptography review, legal drafting, accounting

15.6 Folder structure

One repository. The Flutter and web apps stay where they are; the backend is a single .NET solution with one folder per service. **NEW** marks what v2 adds; Phase 1 files are already in place.

```
khazana/                                  repository root
├── lib/                                  Flutter app
│   ├── core/  auth/ crypto/ sync/   NEW  sign-in + DPoP, keys + sealing, sync engine
│   ├── data/  database/ remote/          Drift v7 (built) · NEW generated API client
│   ├── domain/services/                  notification_parser, capture_router,
│   │                                     capture_account_matcher (built)
│   └── features/  capture/               built · NEW accounts/ cards/ connections/ security/
├── android/app/src/
│   ├── main/kotlin/.../                  MainActivity, CaptureBus, SmsCapture (built)
│   ├── play/                             inert SmsCaptureImpl — no SMS code (built)
│   └── sideload/                         SmsReceiver, SmsCaptureImpl, manifest (built)
├── ios/Runner/Intents/              NEW  SendToKhazanaIntent.swift (P3)
├── webapp/                               Next.js + Tauri · NEW src/lib/api, src/lib/sync
├── backend/                         NEW  .NET 10 microservices
│   ├── Khazana.sln
│   ├── Khazana.AppHost/                  .NET Aspire: runs everything locally
│   ├── Khazana.ServiceDefaults/          OpenTelemetry, health checks, resilience
│   ├── Khazana.Gateway/                  YARP: routes, JWT + DPoP, rate limits
│   ├── services/
│   │   ├── Identity/                     each service has the same five folders:
│   │   │   ├── Khazana.Identity.Api/            minimal API endpoints, DI
│   │   │   ├── Khazana.Identity.Domain/         entities, rules (no framework)
│   │   │   ├── Khazana.Identity.Application/    handlers, validators
│   │   │   ├── Khazana.Identity.Infrastructure/ EF Core, outbox, adapters
│   │   │   └── Khazana.Identity.Tests/          unit + Testcontainers + contracts
│   │   ├── DevicesKeys/  Sync/  AccountAggregator/  Credit/
│   │   ├── MarketData/   Catalog/  Notification/  AuditReporting/
│   │   └── Admin/                        Blazor Server app (reads AuditReporting)
│   └── building-blocks/
│       ├── Khazana.Messaging/            event contracts (versioned), bus setup
│       ├── Khazana.Outbox/               outbox relay + processed-messages
│       ├── Khazana.Crypto/               sealing, signing (NSec), KMS wrappers
│       └── Khazana.SharedKernel/         money (decimal), clock, results, errors
├── contracts/                       NEW  openapi/ per service · events/ schemas
│                                         fixtures/ shared parser cases
├── infra/                           NEW  Terraform: modules/ (network, rds, ecs,
│                                         mq, s3, kms, waf) · envs/ (dev, staging, prod)
├── docs/  adr/  v2-design/               ADRs (NEW) · source of this document
└── .github/workflows/                    ci.yml · NEW one backend job per service
```

15.7 Testing strategy

- **Parser fixtures:** `test/fixtures/parser_cases.json` (built) runs against the Dart and TypeScript parsers, and later the C# templates; grow it with anonymised real alerts toward the top 25 banks.
- **Domain parity:** mirrored tests continue for scoring, dedupe and ledger rules.
- **Per service:** xUnit unit tests; integration tests against real Postgres and RabbitMQ (Testcontainers); **contract tests** for every event and API between services; saga tests that replay each failure path; architecture tests for boundaries.
- **Sync:** property tests for merge convergence; two-device integration tests.
- **End-to-end:** Flutter `integration_test` and Playwright for sign-in, capture, review, undo and sync, run against the Aspire stack.
- **Load and security:** sync and webhook bursts at 10× expected peak; dependency and secret scanning; external VAPT before each phase that adds server surface.

16 Decisions

Decide these by the end of Phase 0, except D13, which can wait until late 2027. Each has a recommendation; D1 and D15 are decided.

#	Decision	Recommendation	Why / alternative
D1	Backend language and architecture	**Decided:** C# / .NET 10, as ten microservices behind a YARP gateway (§7.2)	Your strongest language, and microservices are a learning goal. Costs accepted: about 3–4 more months and higher hosting.
D2	Cloud provider	AWS Mumbai	Mature managed services, KMS, India region. Azure or GCP India are equal alternatives.
D3	Identity	Issue our own tokens after verifying Google and Apple ID tokens	No per-user fees, full control. Firebase Auth is faster to start but adds a dependency.
D4	AA partner and regulatory model	Regulated partner as FIU, Khazana as TSP (model A)	The only realistic path without our own licence. Compare Setu, Finvu and OneMoney quotes.
D5	Gmail reading	On the device	The server never holds mailbox access. Trade-off: syncs only when the app runs or background fetch fires.
D6	Auto-post default	**Built:** on for score ≥ 80, with an "Always review" switch	Meets the "reduce manual work" goal; every automatic add is announced with Undo.
D7	iOS automation	Gmail + OCR + AA at launch, Shortcuts SMS as a later experiment	iOS has no SMS API
D8	Card recommendations	Savings on existing cards only	Recommending new cards is a referral or upsell, against the non-goals
D9	Web scope in v2	Sync and read-mostly views first. Full parity later.	Capture happens on phones. Focus effort there.
D10	Price data	End-of-day only	Real-time licences cost many times more for little user value
D11	Commercial app shielding (RASP)	Start with built-in hardening and attestation. Add a commercial RASP if the mobile pentest finds gaps.	Keeps cost down without lowering the bar. The pentest decides.
D12	Post-quantum encryption	Design for it now (`key_version`), switch on hybrid X25519 + ML-KEM once libraries are stable on all platforms	Financial history must stay private for decades
D13	Monetisation model	Open. Options: B2B version, a paid online tier covering server and partner costs (offline app stays free), or a sponsor.	Decide before AA and credit reports go live, because those cost money per user
D14	Messaging library	Wolverine (MIT) — or MassTransit v8 if you prefer its maturity	Both do outbox and sagas on RabbitMQ; avoid MassTransit v9 (commercial licence since 2025)
D15	Paid services	**Decided:** free first, behind adapters (§13.3)	Find the real issues on free tiers and sandboxes before paying for anything

17 Risks

Risk	Likelihood	Impact	Mitigation
Play rejects the SMS permission	Medium	High	Notification capture already works. Gmail and AA cover the rest. Submit early in Phase 1.
No regulated FIU partner on workable terms	Medium	High	Phases 1–2 don't depend on it. Approach several partners in parallel from Phase 0.
Partner costs higher than estimated	Medium	Medium	Weekly fetch, inactive-user skip, quarterly credit refresh, budget alarms
Bank SMS and email formats change	High	Medium	Server-updatable signed templates, fixture library, balance drift detection
Wrong auto-posted amounts erode trust	Medium	High	Conservative threshold, forced-review rules, Undo, edit tracking for tuning
Google CASA or verification delays	Medium	Medium	Start verification at Phase 0. Ship SMS and OCR first.
Key loss (user loses devices and recovery key)	Low	High	Clear setup flow, recovery key reminders, encrypted local export
Distributed complexity for one person (microservices)	Medium	Medium	Aspire for one-command local runs; traces with a correlation id across every hop; coarse services; shared building blocks; contract tests; build in the order of §15.2 so the platform is proven on a simple service first
Solo burnout, or the project depends on one person	Medium	High	Phases that ship on their own, a sustainable weekly rhythm, ADRs and docs so a collaborator can join, the 36-month fallback plan
Security incident or breach	Low	Critical	§14 baseline: end-to-end encryption, device-bound tokens, HSM keys, VAPT, bug bounty, 6-hour CERT-In response plan, quarterly drills
Funding stops	Medium	High	Offline-first means the app keeps working. Written shutdown plan: 90 days' notice and export.

A Glossary

Term	Meaning
AA	Account Aggregator: RBI-licensed entities that move financial data between institutions with the user's consent
FIP / FIU	Financial Information Provider (for example a bank) / Financial Information User (the regulated receiver of data)
TSP	Technology Service Provider: integrates AA on behalf of an FIU
UTR	Unique Transaction Reference: the 12-digit UPI or bank reference number
DEK / KEK	Data Encryption Key / Key Encryption Key (wraps the DEK)
Sealing	Encrypting a payload to a recipient's public key so only they can open it
LWW	Last-write-wins merge, the existing sync rule
CASA	Cloud Application Security Assessment: Google's required security review for restricted scopes
DPDP	India's Digital Personal Data Protection Act, 2023
NAV / EOD	Net Asset Value (mutual funds) / End-of-day price
Microservice	A small service with its own code, database and deployment, talking to others only through APIs or messages
YARP	Microsoft's reverse-proxy library, used as Khazana's API gateway
.NET Aspire	Microsoft's tooling to run and observe a multi-service app locally with one command
Outbox	Writing an event in the same database transaction as the change it announces, then publishing it — so no event is lost if a crash comes between
Idempotent consumer	A message handler that ignores a message it has already processed, so a redelivery can't apply twice
Saga	A long-running workflow across steps or services, saved as a state machine, with a compensating action for each step that can fail
DPoP	Demonstrating Proof of Possession: tokens bound to a device key, so a stolen token is useless elsewhere
MASVS / ASVS	OWASP's verification standards for mobile apps and for web applications and APIs
RASP	Runtime Application Self-Protection: in-app defence against tampering, hooking and debugging
HSM	Hardware Security Module: tamper-resistant hardware that holds keys and never lets them out
VAPT	Vulnerability Assessment and Penetration Testing

B Sources

Checked in October 2026. Prices from third-party or older documents are marked as estimates in the text. Get current quotes before budgeting.

- Fold: how credit card data is fetched (Gmail) — https://help.fold.money/hc/en-us/articles/17257827045138
- Fold: credit card known issues — https://help.fold.money/hc/en-us/articles/17396989046162
- Fold: introduction to AA — https://docs.fold.money/introduction-to-aa
- Sahamati: data types available on AA — https://sahamati.org.in/data-fi-types-available-on-aa/
- Signzy: Account Aggregator APIs (eligibility, sandbox, certification) — https://docs.signzy.com/api-marketplace/why-use-the-aa-framework
- Federal Bank: Account Aggregator (FIU definition) — https://federal.bank.in/account-aggregator
- Hyperverge: AA framework guide and AA selection guide — https://hyperverge.co/blog/account-aggregator-framework-rbi/
- OneMoney business summary (per-account pricing, older document) — https://www.onemoney.in/docs/resources/FinTech%20Group%20Very%20Brief%20Business%20Summary.pdf
- Setu: Account Aggregator API — https://setu.co/credit-infrastructure/credit-intelligence/account-aggregator-api
- Bajaj Broking: RBI AA interoperability rules — https://www.bajajbroking.in/share-market-news/rbi-account-aggregator-rules-what-interoperability-means-for-your-finances
- Productgrowth: Experian India and CIBIL pricing range — https://productgrowth.in/tools/kyc-identity/experian-india/
- FinBox: credit bureau API providers in India — https://research.finbox.in/blog/best-credit-bureau-data-api-providers-india/
- Decentro: credit bureau API — https://decentro.tech/resources/credit-bureau-api
- Marketcalls: NSE datafeed vendor pricing — https://www.marketcalls.in/?p=12086
- Google Play Console Help: Use of SMS or Call Log permission groups ("SMS-based money management" exception) — https://support.google.com/googleplay/android-developer/answer/10208820
- AWS Free Tier changes for new accounts (credits, 6-month Free plan) — https://spot.rackspace.com/blog/aws-free-tier
- Neon pricing (free plan) — https://neon.com/pricing
- CloudAMQP plans (Little Lemur free tier) — https://cloudamqp.com/pricing
- Grafana Cloud free tier — https://grafana.com/products/cloud/free-tier/
- Finvu sandbox — https://finvu.github.io/sandbox/
- Setu Account Aggregator quickstart — https://docs.setu.co/data/account-aggregator/quickstart
- Khazana v1 repository: docs/ARCHITECTURE.md, docs/APP-INVENTORY.md and the files cited in §3